

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORIES 1 AND 2

Linux Resource Management and xv6 System Calls

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 LABORATORY 1: THE OS AS A RESOURCE MANAGER 5

1.1 Command-to-observation guide 5

1.2 Environment and identity 5

1.3 Files, directories, ownership, and permissions 6

1.4 Processes and the `/proc` view 7

1.5 Memory 8

1.6 Devices and storage 9

1.7 Lab 1 synthesis 10

2 LABORATORY 2: MEET THE OS YOU WILL BUILD 11

2.1 Command-to-observation guide 11

2.2 xv6 build and boot 11

2.3 Using xv6 as a Unix system 12

2.4 Reading user and kernel code 13

2.5 Adding the `sleep` user program 15

2.6 Build and behavior checks 16

CONCLUSIONS 18

REFERENCES 19

ABSTRACT

These two laboratory exercises examine operating systems from complementary sides. Laboratory 1 observes a Linux virtual machine managing files, processes, memory, and devices through standard command-line tools. Laboratory 2 builds and boots xv6 on emulated RISC-V hardware, explores its shell and user/kernel source split, and adds a small user program that invokes the existing `sleep` system call. The report explains what each command or source change demonstrates, how the system-call path connects an application to kernel code, and how to interpret the resulting evidence. Live readings and xv6 results come from a dedicated Ubuntu VM created for these labs; code explanations are tied to the pinned source revision.

Keywords: operating systems, linux, xv6, system calls, process management

INTRODUCTION

The two labs use different methods to study the same subject [1, 2]. In Lab 1, Linux is treated as a working system whose state can be inspected from the terminal. In Lab 2, xv6 is small enough to build, boot, and read as source code. Together they connect observed behavior to the code that implements it.

The report separates three kinds of statements:

a) command and kernel output copied from the dedicated lab VM;

b) explanations of what those observations mean;

c) source-code facts tied to the pinned xv6 revision;

The report uses the adapted ELSE report class [3]. The class supplies English labels and the mandatory report layout.

The experimental observations were recorded on 28 September 2026. Lab 1 terminal values come from the saved VM transcript [4]. Lab 2 build, boot, shell, and timing results come from the saved run and timing logs [5]. This revision changes presentation and explanation; it does not introduce a new execution or new timing samples.

The Linux observations and xv6 runtime results below were captured from Proxmox VM 204, created from an Ubuntu 24.04 cloud image for these labs. It was not the preconfigured course VM mentioned in the handouts. Source-code explanations refer to the pinned xv6 checkout used for the build.

1 LABORATORY 1: THE OS AS A RESOURCE MANAGER

1.1 Command-to-observation guide

The commands are user-space tools. They request information from the kernel through system calls or read kernel-provided interfaces such as `/proc`; the output is a view of state managed by the OS. Table 1.1 links each command group to the evidence to save and the operating-system concept it illustrates.

Table 1.1 – Lab 1 command-to-observation guide

Resource	Commands	Record and explain
VM context	`whoami`, `uname -a`, `cat /etc/os-release`, `uptime`	Record the account, Ubuntu release, kernel and architecture string, and uptime. These identify which running system produced later measurements.
Files	`pwd`, `ls -la /`, `ls -l note.txt`, `chmod 600 note.txt`	Record the owner, group, mode, and two root directories. Explain how a name maps into the tree and how the kernel checks permission bits on access.
Processes	`ps aux | wc -l`, `ps -p 1 -o pid,ppid,comm,args`, `sleep 300 &`, `/proc/PID/status`, `kill PID`	Record PID 1, the approximate count, and the sleep state. Explain that the kernel creates and schedules processes; `/proc` exposes live kernel state through a file-like interface.
Memory	`free -h`, `/proc/meminfo`, `VmRSS`	Record total, free and available RAM, swap totals, and the sleep process RSS. Explain system-wide accounting versus the resident pages of one process.
Devices and storage	`df -h`, `lsblk`, `ls -l /dev | head`, `mount | head`	Record the root mount source, a block-device relation, and one device node. Explain the difference between a device, a filesystem, and the path where that filesystem is mounted.

1.2 Environment and identity

The first commands establish which account and kernel are being observed and provide basic context about the machine's uptime:

Listing 1.1: Initial Linux environment check

```
whoami
uname -a
uptime
```

Listing 1.2: Observed lab VM identity and kernel

```
$ whoami
ubuntu

$ uname -a
Linux ubuntu 6.8.0-136-generic #136-Ubuntu SMP PREEMPT_DYNAMIC Wed Jul 1 21:53:05 UTC
     2026 x86_64 x86_64 x86_64 GNU/Linux

$ uptime
14:24:47 up 21 min, 0 user, load average: 0.00, 0.00, 0.00

$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS (Noble Numbat)"

$ pwd
/home/ubuntu/os-lab1

$ ls -la /
drwxr-xr-x 107 root root 4096 Sep 28 14:04 etc
drwxr-xr-x  12 root root 4096 Aug  1 12:54 usr
```

The VM is Ubuntu 24.04.4 LTS, running the x86-64 Linux kernel shown above. These values identify the environment for the observations in this chapter.

This context matters because command results such as process count, free memory, and device names depend on the particular VM image and its current state. A useful oral explanation is: `whoami` identifies the current account, `uname` describes the kernel serving it, and `uptime` gives a snapshot of how long that kernel has been running.

1.3 Files, directories, ownership, and permissions

Linux presents files in a directory tree rooted at `/`. The lab creates a working directory, writes a text file, copies it, renames the copy, and removes the renamed file. The commands make changes to directory entries and file contents through the same file-system interface used by ordinary programs. The command names are not the kernel: utilities such as `cp`, `mv`, and `rm` run as user-space programs and ask the kernel to read, create, rename, or unlink entries.

Listing 1.3: File operations and permission inspection

```
mkdir -p ~/os-lab1
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
```

Listing 1.4: Selected output showing file owner, group, and mode

```
$ ls -l
total 8
-rw-r--r-- 1 ubuntu ubuntu 24 Sep 28 14:24 note.txt
-rw-r--r-- 1 ubuntu ubuntu 24 Sep 28 14:24 renamed.txt
$ rm renamed.txt
$ ls -l note.txt
-rw-r--r-- 1 ubuntu ubuntu 24 Sep 28 14:24 note.txt
$ chmod 600 note.txt
$ ls -l note.txt
-rw------- 1 ubuntu ubuntu 24 Sep 28 14:24 note.txt
$ stat -c %U\ %G\ %a\ %n note.txt
ubuntu ubuntu 600 note.txt
```

a) The file owner is `ubuntu`; its group is `ubuntu`. The owner and group columns in `ls -l` show which identity the permission bits apply to;

b) For a regular file, mode 600 is expected to display as `-rw-------`. The first character identifies the file type. The next three characters are owner permissions, the next three are group permissions, and the last three are permissions for other users. The actual listing must be checked because it is the evidence for this VM;

c) Two directories observed directly under `/` are `/etc`, which holds system-wide configuration, and `/usr`, which holds installed programs, libraries, and shared data. Both appear in the captured root listing;

The numeric mode is an octal encoding of three permission sets. Each digit is a sum of read (4), write (2), and execute (1). In 600, the owner digit 6 means read plus write (4 + 2); the group and other digits are 0. In 644, the owner keeps read and write, while group and others receive read access. The lab restores the file to mode 644 after checking mode 600. The change demonstrates that access control is metadata the OS enforces for file operations. The leading dash in `-rw-------` says this is a regular file; a directory begins with `d`. The three permission triplets always belong to owner, group, and other users in that order.

1.4 Processes and the `/proc` view

A process is a running instance of a program with an identifier, execution state, memory mappings, and open resources. A source file is not a process: the source is compiled into an executable, and the kernel creates a process when that executable is started. The lab first lists processes, then starts a long-running `sleep` process in the background so it can be identified and inspected while it exists.

Listing 1.5: Process discovery and state inspection

```
ps aux | head
ps aux | wc -l
ps -p 1 -o pid,comm,args
sleep 300 &
PID=$!
ps -p "$PID" -o pid,stat,comm,args
grep -E '^(Name|State|VmRSS):' /proc/$PID/status
kill "$PID"
```

Listing 1.6: Observed PID 1, process count, and sleep state

```
$ ps aux | wc -l
103

$ ps -p 1 -o pid,ppid,comm,args
    PID    PPID COMMAND         COMMAND
      1       0 systemd         /sbin/init

$ ps -ef | grep "[s]leep"
ubuntu 1784 1750 0 14:24 ? 00:00:00 sleep 300

$ grep -E '^(Name|State|Pid|VmRSS|Threads):' /proc/1784/status
Name:   sleep
State:  S (sleeping)
Pid:    1784
VmRSS:  1964 kB
Threads: 1
```

a) Process 1 has PID 1 and runs `systemd`, also invoked as `/sbin/init`. It is the first user-space process, started by the kernel, and manages system services;

b) The idle process listing returned 103 lines. Since `ps aux` prints a header, this corresponds to about 102 listed processes at that instant. Process state changes while the VM runs, so treat the count as a snapshot;

c) The sleeping process state is `S (sleeping)`. The kernel scheduler has put the process in an interruptible wait until its timer expires or another event occurs; it is not continuously consuming a CPU core;

The process directory under `/proc` is a pseudo-file-system: reading its entries asks the kernel to expose information about live processes. It is not an ordinary directory of stored files on disk. This is one example of Linux making kernel-managed state inspectable through a file-like interface. PID 1 is the first user-space process started by the kernel and commonly manages system services; this VM identifies it as `systemd`. The shell's background marker returns control to the prompt, and `$!` stores the last background PID. `kill` sends a signal, SIGTERM by default, asking the target process to stop.

1.5 Memory

`free -h` summarizes memory at the VM level. The per-process `VmRSS` field reports resident memory, meaning pages of that process currently present in physical RAM. The value can vary with kernel

version, process startup, and measurement timing. The `free` column counts currently unused memory, while `available` estimates how much can be given to new work without swapping, including memory reclaimable from caches. They answer different questions. Swap is backing storage, not extra physical RAM, and accessing it is slower than accessing resident RAM.

Listing 1.7: Memory totals and one process's resident set

```
free -h
grep -E '^(MemTotal|MemFree|SwapTotal|SwapFree):' /proc/meminfo
sleep 300 &
PID=$!
grep '^VmRSS:' /proc/$PID/status
kill "$PID"
```

Listing 1.8: Observed memory totals and resident set

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       407Mi       3.0Gi       976Ki       614Mi       3.4Gi
Swap:             0B          0B          0B

$ grep VmRSS /proc/1797/status
VmRSS: 1968 kB
```

a) The VM reports 3.8 GiB total RAM, 3.0 GiB free, and 3.4 GiB available. The `free` column counts currently unused memory, while `available` estimates how much can be given to new work without swapping, including memory reclaimable from caches;

b) Swap is disk-backed space the kernel may use to hold memory pages that are not currently resident in RAM. This VM has no swap configured: total and free swap both show 0 B;

c) The bare `sleep` process had a resident set of 1968 kB in this capture; an earlier capture measured 1964 kB. It is small because the program waits rather than allocating a large application data set. RSS can differ slightly between observations because process startup and memory accounting are sampled at a particular time;

1.6 Devices and storage

`df` reports mounted filesystems and their capacity. `lsblk` shows block devices and their relationships. `/dev` contains device nodes through which programs can use device drivers. These commands expose different layers: a filesystem is mounted on a path, while a block device may provide the storage beneath it. The `/` row in `df -h` identifies which filesystem serves the root path, `lsblk` shows disks and partitions, and `mount` shows filesystem-to-path attachments.

Listing 1.9: Mounted storage and device interfaces

```
df -h
lsblk
ls -l /dev | head
mount | head
```

Listing 1.10: Observed root filesystem and device node

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        19G  1.9G   17G  11% /

$ lsblk
sda       20G  disk
sda1      19G  part /

$ ls -l /dev | head
total 0
crw-r--r-- 1 root root 10, 235 Sep 28 14:03 autofs
drwxr-xr-x 2 root root 320 Sep 28 14:03 block
drwxr-xr-x 2 root root 80 Sep 28 14:03 bsg
crw-rw---- 1 root disk 10, 234 Sep 28 14:03 btrfs-control
drwxr-xr-x 3 root root 60 Sep 28 14:03 bus
lrwxrwxrwx 1 root root 3 Sep 28 14:03 cdrom -> sr0
drwxr-xr-x 2 root root 3360 Sep 28 14:03 char
crw--w---- 1 root tty 5, 1 Sep 28 14:03 console
lrwxrwxrwx 1 root root 11 Sep 28 14:03 core -> /proc/kcore

$ mount | head
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,relatime,size=1985472k,nr_inodes=496368,mode
   =755,inode64)
```

a) The root filesystem at `/` is mounted from `/dev/sda1`, according to `df`. The `lsblk` output shows that `sda1` is a partition on the 20 GiB `sda` virtual disk;

b) The observed `/dev/console` is a character device that provides the system console interface. Its leading `c` in `ls -l` distinguishes it from a regular file; the major and minor numbers `5, 1` route access to its kernel driver;

c) In this context, “everything is a file” means many resources, including device interfaces and kernel-provided information, can be accessed through familiar file operations and paths. It is a useful interface idea, not a claim that every OS resource is literally a regular disk file;

1.7 Lab 1 synthesis

The VM showed four resource-management roles. Filesystem commands such as `ls -l` exposed names, ownership, and access permissions. Process commands such as `ps` and `/proc` exposed running work and its state. `free -h` showed memory and swap accounting. `df`, `lsblk`, and `/dev` showed mounted storage and device interfaces. The outputs recorded above are the evidence; the explanations connect each output to the OS responsibility it demonstrates.

2 LABORATORY 2: MEET THE OS YOU WILL BUILD

2.1 Command-to-observation guide

Here the commands cross layers. The handout's `make qemu` target builds the kernel, user programs, and xv6 disk image, then launches the RISC-V machine in QEMU. Our automated runner performed the corresponding build and launch as two explicit steps so it could capture serial output and command timings. Once the xv6 shell appears, its prompt is a shell running inside the guest OS. Table 2.1 shows what each step demonstrates.

Table 2.1 – Lab 2 command-to-observation guide

Action or command	Record	What it demonstrates
Build and launch, equivalent to `make qemu`	PASS: build 2.158 s; guest shell ready 2.459 s after QEMU start	Boot messages include `xv6 kernel is booting` and `init: starting sh`. The prompt shows that the kernel started the first user process and the shell is accepting commands.
`ls`, `cat README`, `echo`, `wc README`	PASS: `ls` listed `sleep`; echo printed `hello xv6`; `wc` returned 48 lines, 334 words, and 2425 bytes	The filesystem image contains executable user programs and a README file; the shell locates and starts programs.
`ls | grep c`	PASS: output contained `cat`, `echo`, `wc`, and `console`	The shell connects two processes through a kernel pipe so the second program reads the first program's output.
Inspect `user/cat.c` and `kernel/sysfile.c`	Source: `sys_read` is at `kernel/sysfile.c:69` in the pinned checkout	The user program asks for a service; the kernel handler implements the privileged side of that service.
`sleep 10`; `sleep`	PASS: `sleep 10` returned after 0.958 s; `sleep` printed `usage: sleep <ticks>` and returned to the prompt	One call exercises the existing timer service. The other checks the user program's argument validation and error path.

2.2 xv6 build and boot

xv6 is a small Unix-like teaching operating system for RISC-V. The lab builds the kernel and user programs, then starts them on emulated hardware with QEMU. The emulator provides a virtual RISC-V machine; xv6 still has to initialize its kernel, devices, file system, and first user process. There are two OS environments during this exercise: the Ubuntu host runs the compiler, `make`, and QEMU; the xv6 guest runs inside the emulator. After the xv6 prompt appears, commands are being handled by xv6. The QEMU exit sequence returns to the Ubuntu host.

Listing 2.1: The handout command and the equivalent automated build

```
# Handout's interactive command: make qemu
# Actual automated build:
make -C /home/ubuntu/os-lab2-run/xv6-riscv -j2 kernel/kernel fs.img
```

After that build, the runner launched `qemu-system-riscv64` with the compiled kernel and `fs.img`,

connected its serial console to a pipe, and sent the required xv6 commands. The exact QEMU arguments are preserved in the runner script and timing record [5]. The interactive `make qemu` target itself was not invoked in this measured run.

The build completed in 2.158 seconds. QEMU started the xv6 guest, and the shell became ready 2.459 seconds after QEMU launch. This distinguishes a successful compile from a successful boot: the `init: starting sh` message and guest prompt show that the kernel started the first user process.

Listing 2.2: Captured xv6 boot sequence

```
Build elapsed seconds: 2.158
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$
```

QEMU is exited with Ctrl-A followed by X when working interactively, as stated in the assignment. In the automated run, QEMU was terminated after all requested shell commands passed.

The local source tree is pinned to xv6-riscv commit `9374395cd3dc` [6]. This revision provides the `sleep(ticks)` system call required by the handout. A newer upstream revision renamed that call to `pause(ticks)`, which would not match the exercise without adapting its source.

2.3 Using xv6 as a Unix system

At the xv6 prompt, the handout runs programs such as `ls`, `cat`, `echo`, and `wc`, and demonstrates a pipeline. The shell parses each command, starts the relevant user program, connects file descriptors when a pipe is present, and waits for child processes to finish.

Listing 2.3: Example xv6 shell commands from the assignment

```
ls
cat README
echo hello xv6
ls | grep c
wc README
```

Listing 2.4: Captured xv6 shell command results

```
$ ls
README          2 2 2425
cat             2 3 35176
echo            2 4 34096
grep            2 6 42520
sleep           2 14 33824
wc              2 18 36128
console         3 23 0

$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,
...

$ echo hello xv6
hello xv6

$ ls | grep c
cat             2 3 35176
echo            2 4 34096
wc              2 18 36128
console         3 23 0

$ wc README
48 334 2425 README
```

The listing above keeps the entries relevant to the exercise; the full `ls` and `cat README` output is in the run log. All five shell commands completed with a passing check.

Three programs included in the image are `ls`, `cat`, and `echo`. A pipe requires processes that can run the producer and consumer, plus a kernel-managed communication channel that transfers bytes from one process to the other. In `user/sh.c`, the shell creates processes, connects their file descriptors to the pipe, and starts the commands. The pipe buffer lives in the kernel, so the producer can write while the consumer reads. Compared with Linux, xv6's shell provides a much smaller command environment, while the Linux shell has access to a full distribution and a broader set of programs. The familiar command names are separate user-space applications in both systems; the shell coordinates them, and the kernel supplies process and pipe services.

2.4 Reading user and kernel code

The xv6 source separates ordinary programs from privileged operating-system code. Files in `user/` contain programs such as `cat`; files in `kernel/` implement scheduling, memory, file systems, devices, traps, and system calls. A user program requests a service through a system-call interface, and the kernel validates and performs that request. The source file itself is not what the shell executes. The compiler turns a user C file into a RISC-V executable, `UPROGS` tells the build to include it, and `mkfs` packs that executable into `fs.img`. When the shell starts the command, xv6 loads that executable as a user process.

Table 2.2 – Distinguishing source code, user applications, and kernel code

Path or symbol	What it is	How to explain it
`user/sleep.c`	C source for a user program	This is a text file edited by the student. It becomes executable code only after the build compiles and links it.
`user/user.h`	User-side function declaration	It tells C code that `sleep(int)` exists and accepts a tick count. A declaration describes the interface; it does not implement the kernel service.
`user/usys.pl` and generated `usys.S`	System-call entry stub	The build runs the Perl script to generate a small assembly wrapper. The wrapper places the system-call number where xv6 expects it and executes the RISC-V `ecall` instruction to enter the kernel.
`kernel/syscall.c`	System-call dispatcher	It maps the system-call number `SYS_sleep` to the kernel handler `sys_sleep`. This is the dispatch step after the user-to-kernel trap.
`kernel/sysproc.c:66`	The `sys_sleep` handler	It reads the integer argument, checks for a negative value, and waits until the global tick count has advanced by the requested number.
`kernel/proc.c:540`	Kernel scheduler's internal `sleep(chan, lock)` function	This is not the user system call with the same name. It blocks the current process on a kernel wait channel and lets the scheduler run another process.
`Makefile`, `UPROGS`, and `fs.img`	Build and guest filesystem registration	The `$U/_sleep` entry makes the compiled application part of the guest image. Without it, a shell command named `sleep` would not be present in the xv6 filesystem.

The source-level distinction is: `user/sleep.c` is the program source, the compiled `sleep` binary is the application that runs in user mode, and `sys_sleep` is the kernel-side handler. The generated stub connects the two. For `sleep(10)`, the user wrapper executes `ecall`; the trap enters the kernel and the dispatcher selects `sys_sleep`. That handler reads the tick count and repeatedly calls the kernel scheduler's internal `sleep(&ticks, &tickslock)`. A timer interrupt increments the tick counter and wakes processes waiting on the ticks channel. The handler checks the elapsed ticks, returns to user mode after ten ticks, and the program exits. Both the user system call and the scheduler helper are named `sleep`, but their source files, privilege levels, and function signatures show that they do different jobs.

In `user/cat.c`, the important calls have these roles:

a) `open` asks the kernel to open a named file and return a file descriptor;

b) `read` asks for bytes from a file descriptor;

c) `write` asks the kernel to send bytes to a file descriptor, such as standard output;

d) `close` releases a file descriptor when it is no longer needed;

e) `exit` ends the process with a status, while `fprintf` formats an error message in user space and writes it to the selected descriptor;

File descriptors are small integers in a process's descriptor table. By convention, 0 is standard input, 1 is standard output, and 2 is standard error. The same `read` and `write` calls can work with a regular file, a pipe, or a device descriptor, which is how programs can use a common interface for different I/O sources. `fprintf` is a C library function, not a direct system call: it formats text and eventually writes bytes to a descriptor.

The handler `sys_read` is implemented in `kernel/sysfile.c` at line 69 in the pinned checkout. That handler obtains the requested file descriptor and byte count, then delegates the actual read to the kernel's file layer. The wrapper visible to user code and the handler in the kernel are different parts of one request path.

2.5 Adding the `sleep` user program

The new program accepts exactly one argument. It converts that argument to a tick count, invokes xv6's existing `sleep` system call, then exits with status 0. The handout supplies this minimal implementation; it checks the argument count but does not validate the numeric string or propagate a failed system-call return. If the argument is missing or extra arguments are supplied, it prints a usage message to standard error and exits with status 1.

Listing 2.5: user/sleep.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
  exit(0);
}
```

The `argc != 2` check enforces the interface before indexing `argv[1]`. `atoi` converts the decimal argument into the integer expected by `sleep`. The call crosses from user mode into the kernel through xv6's system-call entry mechanism. When the timer count reaches the requested number of ticks, the call returns and the program exits. The argument is measured in kernel clock ticks, not seconds, so `sleep 10` requests ten timer ticks. The exercise checks the number of arguments. In this implementation, xv6's `atoi` stops at a non-digit and does not report conversion errors. Thus a non-numeric argument can become zero; negative

input also becomes zero with this converter. The kernel independently clamps a negative integer argument to zero. These are limitations of the supplied exercise program, so the measured success case uses a positive decimal value. A killed process can cause the kernel call to return -1, but this minimal program still exits with status 0.

The build system must include the new program in `UPROGS`. The entry uses the same `$U/_name` convention as the existing user programs. Without that entry the source file may compile outside the normal image build, but the program will not be copied into the xv6 file system image and will not be available at the shell prompt.

2.6 Build and behavior checks

The assignment checks that the system boots with the program included, that `sleep 10` waits and returns to the prompt, and that calling `sleep` without an argument prints the usage line and returns to the prompt. The source calls `exit(1)` on this error path; xv6's shell does not print a process status value for this command.

Table 2.3 – Required runtime checks and observed outcomes

Command	Expected behavior from the handout	Observed VM result
Build and launch (handout: `make qemu`)	Kernel boots and xv6 shell prompt appears	PASS: equivalent explicit build completed in 2.158 s; shell was ready 2.459 s after QEMU launch.
`sleep 10`	Waits for ten ticks, then returns to prompt	PASS: returned after 0.958 s in the measured run.
`sleep`	Prints the usage line and returns to the shell; the source exits with status 1	PASS: printed `usage: sleep <ticks>` and returned to the prompt.

Listing 2.6: Captured sleep program tests

```
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

The timing record measured `sleep 10` at 0.958 s. In this pinned kernel, the timer interval is nominally 100 ms, so ten ticks are approximately one second. The user call starts within the current tick interval; prompt detection and emulator scheduling also affect the measured wall-clock duration. This single observation does not establish a precise conversion factor or a benchmark distribution. The bare `sleep` test printed its usage line and returned to the shell; `exit(1)` is confirmed by the program source, not by a status value printed in the xv6 shell.

The source change is present in the local xv6 tree: `user/sleep.c` implements the argument check

and system call, and `Makefile` registers `$U/_sleep`. The build included `sleep` in the xv6 image, and the guest shell successfully executed it with and without an argument.

CONCLUSIONS

Lab 1 made the OS visible through its interfaces: file metadata, process state, memory accounting, mounts, and device nodes. Lab 2 followed a request in the other direction, from a user program through a system call into kernel code. The new `sleep` program is short because the kernel already provides the timer-based service; the user program supplies argument handling and asks the kernel to perform the wait. This distinction between policy and mechanism, user code and privileged code, is central to understanding how operating systems organize services.

Each explanation is tied to a concrete observation: a permission listing, process state, memory field, mounted-device row, source location, or xv6 runtime transcript. The lab results show how OS concepts move from command output to source code and then to a working program.

REFERENCES

1. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova. *Operating Systems: The OS as a Resource Manager, Lab 1, Track A*. Course handout, 2026–2027.

2. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova. *Operating Systems: Meet the OS You Will Build, Lab 2, Track B*. Course handout, 2026–2027.

3. Universitatea Tehnică a Moldovei. *ELSE mandatory report template*. Supplied Word model and adapted UTM academic paper template repository, updated 2 October 2026. Accessed 2 October 2026. `https://github.com/Lucian-Adrian/utm-academic-paper-template`.

4. Dedicated lab VM 204. *Lab 1 Linux terminal capture*, `D:/ops/uni/os/lab1/evidence.txt`, 2026-09-28.

5. Dedicated lab VM 204. *Lab 2 xv6 run log and timing records*, `D:/ops/uni/os/lab2/lab2-run.log`, `D:/ops/uni/os/lab2/lab2-timing.jsonl`, 2026-09-28.

6. MIT PDOS. *xv6-riscv source code*, commit `9374395cd3dc`. `https://github.com/mit-pdos/xv6-riscv`.